Ternary Neural Network Inference

for Handwritten Digit Recognition on an FPGA

A multiplier-free $784 \to 32 \to 10$ ternary network,
verified bit-exact against its software reference

Project Report

Contents

1 Introduction **3**
- 1.1 Problématique . . . 3
- 1.2 Proposed solution . . . 3
- 1.3 Why ternary networks . . . 3
- 1.4 Scope and honest positioning . . . 4
- 1.5 Contributions . . . 4

2 Ternary Compression in the Wider AI Landscape **5**
- 2.1 Why the pressure is structural . . . 5
- 2.2 Large language models . . . 5
- 2.3 Edge, embedded and always-on AI . . . 6
- 2.4 Vision, generative models and multimodal systems . . . 6
- 2.5 Robotics, autonomy and certification . . . 6
- 2.6 Hardware directions . . . 6
- 2.7 What has to be true for the transition . . . 7
- 2.8 A miniature of the same problem . . . 8

3 Related Work: Classical Reference **8**
- 3.1 Architecture . . . 8
- 3.2 The information-content argument . . . 8
- 3.3 Cost model . . . 9

4 Preliminaries: Ternary Weight Compression **9**
- 4.1 Weight encoding and the cost of a ternary weight . . . 9
- 4.2 Measured bits per weight . . . 9
- 4.3 Techniques for rigidity and accuracy . . . 10
- 4.4 The rotation in detail . . . 11

5 Proposed Methodology **11**
- 5.1 Architectural decisions . . . 11
- 5.2 Integer-exact derivation . . . 11
- 5.3 Choosing the hidden width . . . 12
- 5.4 The accuracy–robustness trade . . . 12

6 Hardware Implementation **13**
- 6.1 Design overview . . . 13
- 6.2 Weight memory organisation . . . 13
- 6.3 Resource usage . . . 13
- 6.4 Timing . . . 13

7 VHDL Components **14**
- 7.1 Hierarchy: components and subcomponents . . . 15
- 7.2 Board-level components . . . 17
  - 7.2.1 `Nexys3Top.vhd` — board wrapper . . . 17
  - 7.2.2 `UARTReceiver.vhd` . . . 18
  - 7.2.3 `ImagePacketReceiver.vhd` . . . 18
  - 7.2.4 `SevenSegmentDisplay.vhd` . . . 19
- 7.3 The classifier datapath . . . 19
  - 7.3.1 `NumberRecognition.vhd` — classifier datapath . . . 19
  - 7.3.2 `WalshHadamard.vhd` — block-8 rotation . . . 20
- 7.4 The multiplier-free MAC blocks . . . 21
  - 7.4.1 `TernaryMacEngine.vhd` — the sequential MAC . . . 22
  - 7.4.2 `NeuronLayer.vhd` — the hidden-unit array . . . 22
  - 7.4.3 `TernaryNeuron.vhd` — one hidden unit . . . 23
  - 7.4.4 `TernaryApply.vhd` — the ternary multiply . . . 23
  - 7.4.5 `Adder.vhd` — the adder . . . 24
  - 7.4.6 `Accumulator.vhd` — the accumulator . . . 24
  - 7.4.7 `ReLU.vhd` — the activation . . . 25
  - 7.4.8 `Saturate.vhd` — ReLU plus saturation . . . 25
- 7.5 Output layer and class selection . . . 26
  - 7.5.1 `DenseLayer.vhd` — combinational output layer . . . 26
  - 7.5.2 `ArgMax.vhd` — class selection . . . 26

7.6 `ModelWeights.vhd` — generated constants . . . 27
7.7 Testbenches . . . 27
8 Verification Methodology **28**
8.1 The bit-exactness principle . . . 28
8.2 Verification stages . . . 28
8.3 Defects discovered . . . 28
9 Results **29**
9.1 Accuracy . . . 29
9.2 Complexity analysis . . . 29
9.3 Energy . . . 30
9.4 Result figures . . . 31
9.5 Simulation results . . . 32
9.6 Confusion analysis . . . 32
10 Negative Results **33**
11 Implementation Roadmap **34**
11.1 Outstanding tasks . . . 35
11.2 The one known synthesis risk . . . 35
12 Limitations **36**
13 Conclusion **36**

Abstract

This report describes the design, implementation and verification of a ternary ($\{-1, 0, +1\}$) neural network accelerator for handwritten digit recognition on the Digilent Nexys 3 FPGA (Spartan-6 XC6SLX16). The deployed network is $784 \to 32 \to 10$: its first layer is entirely ternary and contains *zero* arithmetic multipliers, because multiplication by -1, 0 and $+1$ reduces to a sign flip, a mask and a wire.

The work is organised around a single principle: **bit-exactness**. The FPGA must reproduce the PyTorch model exactly, not approximately. This forced the entire forward path to remain in integer arithmetic, and every architectural decision follows from that constraint rather than from convenience. A Walsh–Hadamard rotation is applied to the pixel bus before the first layer so that the ternary MAC never accumulates beyond a narrow integer range; the training procedure is run in an `integer-exact` mode that reproduces the hardware's fixed-point arithmetic exactly during optimisation.

Nine classes of defect were found and fixed during verification, several of which had produced plausible-looking but silently wrong results. Final verification reports **2000/2000 agreement** between the RTL simulation and the software reference over 2,000 MNIST images.

Against a classical float reference of the same task, the ternary design achieves a **complete elimination** of the multiplier count and a **228**× reduction in the information content of the weights — 115× fewer bytes stored — for a loss of 3.4 accuracy points on the test set. We report this honestly as a *cost* trade, not a free win: the float reference remains more accurate. The value proposition is area, latency and energy, which are the metrics that matter for an edge classifier.

1 Introduction

1.1 Problématique

Embedded neural-network inference is constrained by three resources: silicon area, latency and energy. The dominant term in all three is the multiply–accumulate (MAC) unit. A conventional fully-connected layer with n inputs and m outputs requires nm multipliers, each occupying on the order of a thousand LUTs on a 7-series FPGA. A $784 \to 256$ layer therefore cannot fit on the 15,850 LUTs available on a Nexys 3. The problem addressed by this work is: *how to deploy a useful MNIST classifier on a device that cannot hold a single conventional hidden layer, without sacrificing the ability to verify the result.*

The two candidate devices are worth stating precisely, because the report names one and the board is the other. The Digilent Nexys 3 carries a Spartan-6 XC6SLX16: 15,850 LUTs, 32 BRAM18 blocks (520 Kib) and 32 DSP48E1 slices. The Artix-7 XC7A100T has 63,400 LUTs, 135 BRAM36 blocks (4.86 Mb) and 240 DSP48E1 slices. Every device budget in this report is quoted against the *smaller* Spartan-6 part, which is the conservative choice and the one the shipped constraint file targets. Task 0 of the roadmap (Section 11) resolves the naming question; nothing in the argument below depends on which way it is resolved.

1.2 Proposed solution

The proposed solution is a multiplier-free ternary inference datapath. By restricting the first-layer weights to $\{-1, 0, +1\}$, multiplication collapses into a sign flip, a mask or a wire, eliminating all DSP usage in the dominant layer. A Walsh–Hadamard rotation of the pixel bus keeps the accumulator narrow, and an `integer-exact` training mode ensures the RTL reproduces the PyTorch model bit-for-bit. The design is $784 \to 32 \to 10$, stores its weights in 3.5 KiB of information, 6.9 KiB of packed constants, performs zero multipliers in the first layer, and classifies in 15.7 µs of compute (17.8 µs including framing) at 100 MHz, at 94.61% MNIST accuracy.

1.3 Why ternary networks

Neural network inference on an embedded device is constrained by three resources: silicon area, latency and energy. The dominant term in all three is usually the multiply–accumulate unit. A conventional layer of n inputs and m outputs requires nm multipliers, each occupying on the order of a thousand LUTs on a 7-series FPGA. A $784 \to 256$ layer therefore does not fit on the 15,850 LUTs available on a Nexys 3.

Table 1: The three ternary weights and what each costs in hardware. Only the first row requires no multiplier, which is the entire point of the exercise.

Weight	Operation	Hardware	Multiplier
$+1$	$\times 1$	pass the wire	no
0	$\times 0$	mask to zero	no
-1	$\times -1$	invert the sign bit	no

Table 1 is the whole argument. If every weight in the large first layer is drawn from $\{-1, 0, +1\}$, the entire $784 \times H$ multiply stage collapses into wiring. What remains is an adder tree and a weight memory, and the layer becomes cheap enough that several times more hidden units fit.

This is not a new idea. Ternary weights were introduced as Ternary Weight Networks [2] and trained ternary quantisation [3], and integer-only inference was formalised by Jacob *et al.* [4]. BitNet b1.58 and the Bonsai 27B work establish that fully ternary networks are competitive with much larger floating-point ones at inference, and this project draws its technique set from that lineage (Section 4). What is new here is the *integer-exact hardware co-design*: the network is

constrained during training so that the RTL is not an approximation of the model but a faithful implementation of it.

1.4 Scope and honest positioning

The classical float network used as reference is the 784–256–10 teacher from this project, reaching **98.05%** on the MNIST test set. The ternary network reaches **94.61%**. The ternary design does not win on accuracy and this report does not claim that it does.

The comparison that matters is cost. Table 2 summarises the positioning, and Section 9 develops it in full.

Table 2: Headline comparison. The float reference is more accurate; the ternary design is far cheaper. Every figure in this table is measured, not estimated. "Information" is the entropy of the weights; "encoded" is what the hardware actually holds, at two bits per first-layer weight. The BRAM36 row counts the encoded size in 36 Kb blocks.

	Float ref. 784–256–10	**Ternary** H=16 784–16–10	**Ternary** H=32 784–32–10
MNIST test accuracy	98.05%	94.19%	94.61%
MNIST (2,000 pipeline)	97.0%	92.5%	93.5%
Hand-drawn	80.5%	74.8%	84.8%
Rendered font glyphs	91.1%	78.9%	83.3%
Parameters	203,530	12,730	25,450
Multipliers	203,530	0	0
MACs per image	203,264	12,704	25,408
Weight storage (information)	795.0 KiB	1.8 KiB	3.5 KiB
Weight storage (encoded)	795.0 KiB	3.5 KiB	6.9 KiB
BRAM36 for weights	177	0.8	1.5
Entropy (bits/weight)	32.0	1.189	1.122

Two points deserve emphasis, both of which cut against the project's own interest. First, the float reference is more accurate on every input tested and this report does not pretend otherwise. Second, Section 10 documents five substantial optimisation attempts that failed, one of which made accuracy actively worse.

1.5 Contributions

1. A multiplier-free ternary inference datapath for $784 \to 32 \to 10$, with a sequential MAC engine sized to fit the Nexys 3 LUT budget.

2. An `integer-exact` training mode in which the forward pass is written to reproduce the hardware's integer arithmetic exactly, so the exported weights are provably faithful rather than approximately faithful.

3. A verification methodology that compares the RTL against the software reference image-by-image, reported at 2000/2000 agreement.

4. A host toolchain: a training GUI with live network visualisation, an image uploader with automatic polarity detection, and UART transmission to the board.

5. A documented negative-result record covering distillation, ensembling, test-time augmentation, high-resolution drawing and per-digit rebalancing (Section 10).

2 Ternary Compression in the Wider AI Landscape

The network in this report has 25,450 parameters, of which 25,408 are weights. That is six orders of magnitude smaller than a modern language model, and it would be easy to read the project as a curiosity that does not generalise. The opposite is true. The constraint that shaped this design — *move fewer bits, and do not multiply* — is now one of the organising constraints of artificial intelligence as a whole. This section places the project in that trajectory, and is explicit about which claims are shipping practice and which are research.

2.1 Why the pressure is structural

Three quantities have grown at different rates over the last decade: model size, memory bandwidth, and the energy budget available at the point of use. Model size has grown fastest. The consequence is that inference on a large model is usually *memory-bound*, not compute-bound: the arithmetic per weight is small, so the cost of fetching the weight dominates the cost of using it [1]. Every bit removed from a weight is therefore removed from the critical path of every token, every frame and every inference.

This is why compression is not a temporary optimisation. A factor of two in weight volume is a factor of two in the dominant term, and that term does not disappear when the process node improves. Ternary weights take the argument to its limit: two bits per weight, of which one encoding is unused, for an entropy of about 1.12 bits per weight in this design (Section 4).

2.2 Large language models

The clearest evidence that ternary weights scale is the BitNet line of work. BitNet b1.58 [5] trains a Transformer whose linear-layer weights are restricted to $\{-1, 0, +1\}$ and reports accuracy matching an FP16 baseline of the same parameter count from roughly three billion parameters upward, while:

- eliminating every floating-point multiplication in the attention and feed-forward projections — only additions and subtractions remain;
- reducing weight memory by about 3.5× relative to FP16, which allows a larger model in the same memory;
- reducing the energy of the linear layers by up to 71× in the authors' arithmetic model, because addition is far cheaper than multiplication and the memory traffic is smaller;
- running about 2.4× faster on CPU, where the bottleneck is memory movement rather than vector throughput.

The mechanism is exactly the one used here. A ternary weight turns a multiply–accumulate into an add, a subtract, or nothing; the accumulator and the memory interface become the whole datapath. Recent 27-billion-parameter ternary models extend the same result to a size where serving cost is a real commercial constraint.

Ternary weights are the far end of a curve that the field has already travelled most of the way along. Post-training quantisation of large models to three or four bits per weight is now routine and is reported to cost little accuracy [6], and mixed-precision recipes that keep activations at eight bits while compressing the weights are the standard way to run these models on accelerators [7]. The division of labour is the same as in this design: compress the large matrix, keep the small one wide.

One caveat matters and is often omitted: these results come from *training* in ternary from the start, not from rounding a finished FP16 model. Post-hoc ternarisation of an existing network degrades badly, which is why this project also trains in an integer-exact mode rather than quantising afterwards.

2.3 Edge, embedded and always-on AI

At the other end of the scale, the same trade decides whether a product exists at all. A microcontroller-class device has tens to hundreds of kilobytes of memory and a power budget measured in milliwatts; a wearable or an implant has less. In that regime a floating-point unit is a luxury and a multiplier is a significant block. Ternary weights fit in on-chip flash, need no DSP slice, and turn the accelerator into an adder tree with a small ROM.

The applications are unglamorous and numerous: keyword spotting, gesture recognition, vibration-based predictive maintenance, seizure and arrhythmia detection, always-on vision wake-up circuits, and sensor fusion in wearables. In each case the model is small, the input rate is low, and the energy per inference is the product specification. A multiplier-free integer datapath is not a compromise there; it is the only way to hit the budget.

This project is a worked example at exactly that scale: 15,850 LUTs, no DSP slice in the first layer, 6.9 KiB of packed weight constants, 15.7 µs per image and a datapath that is bit-exact against its training model. The number that matters for a product is not the 94.61% alone but the combination of accuracy, area and energy that a ternary layer makes possible.

2.4 Vision, generative models and multimodal systems

Attention and feed-forward blocks dominate vision Transformers and diffusion models in the same way they dominate language models, so weight compression transfers directly. The active research questions are different, though. Image and video models are more sensitive to activation precision than to weight precision, which favours the *selective* approach used here: ternary where the weight matrix is large, higher precision where the dynamic range is needed. In this design the second layer is kept at 16 bits for exactly that reason, and it costs 1.26% of the weights.

Two further directions are worth naming. The first is on-device generation: a ternary backbone with higher-precision adapters can fit a diffusion or language model into a phone or a headset, where the alternative is sending every request to a datacentre. The second is conditional computation: a third state is a natural encoding for "skip this block", so ternary logic can gate compute as well as compress weights, which is a distinct source of energy savings from weight compression itself.

2.5 Robotics, autonomy and certification

Robotics combines the edge constraint with a second requirement that software accelerators often ignore: determinism. A controller that occasionally produces a different answer for the same input is difficult to certify, whether the certification regime is automotive, medical or avionics. Integer arithmetic with a fixed order of operations has no such ambiguity. The bit-exact property that this project enforces for verification is, in a certification context, a safety argument: the deployed artefact can be shown to compute exactly the function that was validated.

2.6 Hardware directions

The compression argument changes what hardware is worth building. Table 3 summarises the directions that are currently active, with an honest note on maturity.

Table 3: Hardware directions that exploit ternary or near-ternary weights. "Maturity" distinguishes shipping silicon from research prototypes and analytical proposals.

Direction	Mechanism	Maturity
FPGA adder-tree accelerators	weights in ROM, one input per cycle, no DSP	shipping (this work)
RISC-V vector extensions	ternary dot-product instructions, packed 2-bit operands	research silicon
Compute-in-memory (RRAM, PCM, MRAM)	the ternary weight is the conductance state; the crossbar performs the MAC	research prototypes
Ternary content-addressable memory	three-state match for gating and early exit	shipping (networking), research (AI)
Integer-only NPUs	8-bit or lower activations, high-precision accumulators	shipping
Structured sparsity	zeros map to clock gating and skipped cycles	shipping

Compute-in-memory is the most interesting of these because it removes the memory traffic rather than compressing it: if the weight is a conductance, it does not have to be fetched at all. Ternary states are attractive there because three levels are easy to program and to distinguish, and because the peripheral circuitry can be simple. The catch is the same one this project documents in Section 11: without a synthesis and measurement path, the advantage remains analytical.

2.7 What has to be true for the transition

Ternary compression is not a free lunch, and the report would be dishonest if it implied otherwise. Four conditions have to hold for a system to benefit:

1. **Accuracy must survive.** Locally, this design gives up 3.4 accuracy points on the MNIST test set relative to a float reference (94.61% against 98.05%). At language-model scale the reported gap is smaller, but it is not zero, and it is task-dependent.

2. **Training must be ternary-aware.** Rounding after training is not equivalent; the model has to see the quantiser during optimisation.

3. **The toolchain must exist.** A quantiser without an exporter, a simulator and a bit-exactness check is a research result, not a product component. The work in Section 5 exists because that gap is where most compression projects fail.

4. **The metric must be the right one.** If the deployment target is a datacentre GPU with abundant bandwidth, ternary weights buy little. If it is a battery-powered sensor, they buy the product.

2.8 A miniature of the same problem

The four techniques that made this design work are not specific to MNIST.

- **Rotation before quantisation.** A Walsh–Hadamard transform spreads the input energy so that the accumulator stays narrow. The same rotation, at a larger block size, is what keeps a wide Transformer activation in range.
- **Selective precision.** The 32×10 output layer stays at 16 bits because it is 1.26% of the weights and carries most of the dynamic range. The same argument applies to the output projection and the embedding table of a language model.
- **Closed-loop sparsity.** The threshold is adjusted during training to hold a target zero density (65.86% here), rather than fixed in advance. This is how a ternary model avoids both collapse and useless density.
- **Bit-exact co-design.** The hardware is not an approximation of the model. At any scale, the moment the two are allowed to diverge, the verification argument disappears.

The scale differs by six orders of magnitude; the structure of the problem does not. The Nexys 3 version is small enough to verify completely, which is why it is a useful place to learn what a ternary datapath actually costs — and the same accounting is what a language-model accelerator needs, one thousand times larger.

3 Related Work: Classical Reference

3.1 Architecture

The reference network is a fully connected multilayer perceptron, $784 \to 256 \to 10$, trained in float32 by the original supervision path in this project. It reaches **98.05%** on the 10,000 MNIST test images (its recorded validation accuracy, the split the checkpoint was selected on, is 98.24%). It is used here in two roles: as the accuracy yardstick, and — in one discarded experiment — as a distillation teacher.

Its cost is what motivates everything else. At float32 it holds $203{,}530 \times 32 = 6{,}512{,}960$ bits of weights, or roughly 795 KiB, which is 177 BRAM36 blocks — more than the 135 the XC7A100T has, and eleven times the 16 BRAM36 equivalents of the Nexys 3's Spartan-6. It requires 203,264 multiply–accumulate operations per image and therefore a comparable number of multipliers: as DSP48E1 slices that is 203,264 of them, against the 32 the Spartan-6 has and the 240 the XC7A100T has; built from LUTs it would be several million of them. Neither figure is compatible with a device that also has to run the rotation, the control logic and the UART receiver.

3.2 The information-content argument

The gap between float and ternary is fundamentally a question of information. A float32 weight carries 32 bits; a ternary weight carries at most $\log_2 3 = 1.585$ bits. Compressing a weight by a factor of roughly twenty is a substantial loss of degrees of freedom, and it would be unreasonable to expect it to cost nothing.

The prediction from this argument is that ternary accuracy should be bounded by the float baseline and should approach it only as capacity grows. Section 5.3 tests this directly with a width sweep, and the result is consistent: the measured entropy of the deployed ternary network is 1.12 bits per weight, 3.5% of the float representation, and it recovers 96.5% of the reference's accuracy while using 12.5% of the parameters.

3.3 Cost model

Unless stated otherwise, all cost figures in this report use the following definitions:

Multipliers.
The number of distinct multiply operations the datapath must perform per inference. A ternary weight consumes none, so the count equals the number of float weights.

MACs.
Multiply–accumulate operations per image, counting one MAC per weight application.

Latency.
The number of input cycles divided by the clock frequency. The engine consumes one input per cycle and applies it to all 32 units at once, so the 25,408 MACs of an image cost 784 cycles, not 25,408. For the float reference, whose products are not shared across units, latency is the MAC count divided by the clock frequency. Both figures are cycle-accurate.

Weight storage.
Two figures are reported, and they are not the same thing. The *information content* of the weights is the entropy of the first layer plus 16 bits per second-layer weight; the float reference is counted at its native 32 bits per weight. The *encoded* size is what the hardware actually holds: the first layer is stored in its two-bit packing, one of the four encodings unused, so the encoded size is larger than the information content. Both are stated wherever a storage ratio is quoted.

No *measured* power or area figure is quoted anywhere in this report. No synthesis tool is available in this environment, so such numbers would be fabricated; where an area figure appears (Table 6) it is labelled as an analytical estimate and is replaced by measured counts in task 4 of the roadmap. Energy claims are handled in Section 9.3 by counting arithmetic operations and memory accesses, which *is* measurable, and by stating explicitly which parts of the argument are inference.

4 Preliminaries: Ternary Weight Compression

4.1 Weight encoding and the cost of a ternary weight

A ternary weight is stored in two bits, with the fourth of every four-bit group unused. This is wasteful in isolation — 2 bits per weight where the entropy is 1.19 — but it is what the VHDL address arithmetic wants, because it keeps the word boundaries aligned for direct indexing rather than requiring a shift and mask on every access.

The packed first-layer matrix occupies $784 \times 32 \times 2 = 50{,}176$ bits, which fits in two BRAM36 blocks.

4.2 Measured bits per weight

The meaningful figure is not the 2-bit storage format but the entropy of the learned weights. For a ternary matrix with zero fraction z, the Shannon entropy per weight is

$$H(z) = -z \log_2 z - (1 - z) \log_2 (1 - z).$$

Because every digit class carries roughly the same number of examples, the weights come out close to the maximum-entropy ternary distribution $z = 1/3$, which gives 1.585 bits. Measured across the shipped models:

Table 4: Measured weight entropy. "Zero density" is the fraction of first-layer weights that quantise to exactly zero, the value that makes the hardware a mask-and-add network.

Configuration	Zero density	$H(z)$	Net bits/weight
without augmentation	53.4%	0.993	1.191
wide augmentation	69.3%	0.816	1.086
deployed (mild)	65.9%	0.926	**1.122**

The net figure includes the second layer at its true 16-bit width, which is why it sits above $H(z)$: selective precision is not free, it costs $16 \times 160 = 2{,}560$ bits, but it is only 1.26% of the weights.

4.3 Techniques for rigidity and accuracy

Five techniques were adopted from BitNet b1.58 and Bonsai 27B, and two were developed for this project in response to measured failures. Each is described with the problem it solves, because several of them exist precisely because a naive ternary implementation did not work.

1. Walsh–Hadamard rotation. The dominant error in ternary networks is magnitude growth. When a ternary weight multiplies an unbounded activation, the accumulator drifts far from zero, and the hardware is forced into a wide integer to avoid overflow — destroying the area saving.

The fix, borrowed from Bonsai, is to apply a fixed orthogonal transform H to the input vector before the first layer, and to store $W_1 H^{\mathsf{T}}$ in its place. Since H is orthogonal, the layer computes an equivalent function of a rotated input rather than a different function. Block-8 Hadamard matrices are used: the rotation is a fixed XOR/adder network with no multipliers, and it normalises the input energy so the accumulator stays narrow.

This is the single most important technique in the design. Without it the integer widths grow and the hardware stops being small.

2. Selective precision on the second layer. The output matrix has only $H \times 10$ weights — 320, or 1.26% of the network. Forcing these into $\{-1, 0, +1\}$ as well would be aggressive for almost no area gain. Bonsai's observation is that precision is worth paying for wherever it is cheap, so W2 is kept at 16 bits. This is what allows a 16-ternary-unit network to reach 94% on MNIST: the decoder does the fine discrimination that the encoder cannot.

3. Closed-loop sparsity control. The ternary threshold ratio is not a hyperparameter fixed in advance. It is adjusted during training by measuring the achieved zero density against a target and stepping the ratio accordingly, so the network is driven towards the target sparsity rather than drifting away from it.

4. Positive bias initialisation for dead-unit prevention. With a positive bias initialisation, a hidden unit begins the training in its active state. Without it, a unit whose weights quantise to zero stays inactive, receives no gradient, and is dead for the rest of training. This matters more in ternary networks than in float networks because the quantisation step can kill a unit outright; dead units are unrecoverable at the quantisation boundary.

5. Augmentation to match the deployment distribution. MNIST is pre-normalised: every digit sits in a 20×20 box. A user drawing with a mouse does not. This was measured, not assumed — freehand digits average a 25.3-pixel span against MNIST's 20.0, and carry 186 pixels of ink against MNIST's 96. Training without augmentation on that distribution yields roughly 18% accuracy on freehand input. Scale, translation, rotation and stroke thickness augmentation

recover it, at a cost of roughly 11 points of clean MNIST accuracy. Section 5.4 documents that trade honestly.

6. Integer-exact training (developed here). The usual approach is to train in float and quantise afterwards, accepting that the hardware will differ from the model. That is unacceptable for this project, whose central claim is bit-exactness. Instead the forward pass is written in integer arithmetic that mirrors the RTL: binary pixel inputs, an unnormalised Hadamard rotation with a fixed right shift, integer biases, and an integer activation clamp.

The gradient estimator is the straight-through estimator, which must be written as `hard.detach() + (latent - latent.detach())` rather than the more common `x + (q - x).detach()`. Both produce correct gradients in principle, but the second form silently produces wrong ones when the latent is used more than once in the graph. Two separate defects in this project traced back to that distinction.

7. Adaptive geometry guidance in the host tool (developed here). Because the model is positionally sensitive — a one-pixel shift costs 3 to 9 accuracy points — the GUI measures the drawing and reports its span, centre and ink coverage against the training distribution. The original design imposed a fixed target box and told the user to aim at it; measurement showed that attempting to hit a fixed box by resampling costs about 23 points through binarisation error. Measuring and reporting is strictly better than pretending to correct.

4.4 The rotation in detail

For an input block $\mathbf{x}$ of length 8, the Hadamard transform is the butterfly

$$\begin{bmatrix} x_0 & x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{bmatrix} \mapsto \begin{bmatrix} x_0{+}x_1{+}x_2{+}x_3{+}x_4{+}x_5{+}x_6{+}x_7 & \cdots \end{bmatrix}$$

implemented in `WalshHadamard.vhd` as a streaming XOR/adder network. It is verified bit-identical to the Python reference over 3,136 values with total absolute error zero.

5 Proposed Methodology

5.1 Architectural decisions

The central design decision was to abandon a fully parallel first layer. A parallel 784×16 design needs approximately 25,000 LUTs against the 15,850 available on the device; it does not fit, at any optimisation effort. Replacing it with a single sequential MAC engine that walks the 784 inputs costs one multiplier-equivalent (none, being ternary) and 784 cycles, and fits comfortably.

The cost of this decision is latency, and it is smaller than the MAC count suggests: the 32 units run in parallel, so the 25,408 MACs of an image take 784 cycles, not 25,408. Measured end-to-end, including the rotation, the classifier takes 1,569 cycles — 15.7 µs at 100 MHz. For a human-facing digit recogniser this is entirely irrelevant, and the area saving is what makes the design exist at all.

5.2 Integer-exact derivation

The integer-exact mode constrains the forward pass so that every value the hardware could see is reproducible in Python:

- Pixels are binarised before the rotation, so the rotation input is strictly $\{0, 1\}$.
- The rotation is unnormalised and followed by a fixed right shift, matching `WalshHadamard.vhd` exactly.

- First-layer biases are rounded to integers *before* being divided by the shared scale, not after. This ordering was a genuine defect: the opposite order silently changed results.
- The accumulator is allowed to run freely through the whole MAC pass; ReLU and clamping are applied only on the final cycle. Clamping earlier looked reasonable and was wrong.
- The second layer is scaled by a single positive constant. A shared positive scale leaves the arg-max unchanged, so this is free.

5.3 Choosing the hidden width

Because the information-content argument predicts that ternary networks approach float accuracy only as capacity grows, a width sweep was run to choose H. The result was not the monotone improvement the argument predicts: MNIST accuracy rose with width while robustness to hand-drawn input did not, and the H=32 row was the weakest of the four on average. Width alone therefore does not buy robustness, which is why the deployed model pairs H=32 with augmentation rather than simply being the widest network available.

The decisive evidence for width came from the confusion analysis in Section 9, which shows the dominant $8 \to 6$ error is a loop-count distinction that does not clear until there are enough hidden units to represent the feature.

5.4 The accuracy–robustness trade

Augmentation is not free. Table 5 shows the cost precisely.

Table 5: Effect of augmentation, measured on the models of the width sweep, where the trade is cheapest to see. Freehand accuracy is measured on the synthetic freehand set; MNIST on the real test set.

Training	MNIST	Freehand
No augmentation	94.19%	18.4%
Morphology only	91.78%	27.2%
Morphology + affine	83.96%	68.4%

Augmentation moved freehand accuracy from 18.4% to 68.4% — the difference between unusable and usable — for 10 points of clean MNIST accuracy. Whether that is a good trade depends entirely on the deployment distribution. For a benchmark submission the answer is clearly no; for an interactive drawing tool it is clearly yes. Both models are retained.

The deployed network takes the middle of that trade rather than either end of it: a mild augmentation that costs less than two points of MNIST accuracy (94.61% on the test set, 93.5% through the pipeline) and buys 84.8% on hand-drawn input, well above both the unaugmented and the wide-augmentation variants.

6 Hardware Implementation

6.1 Design overview

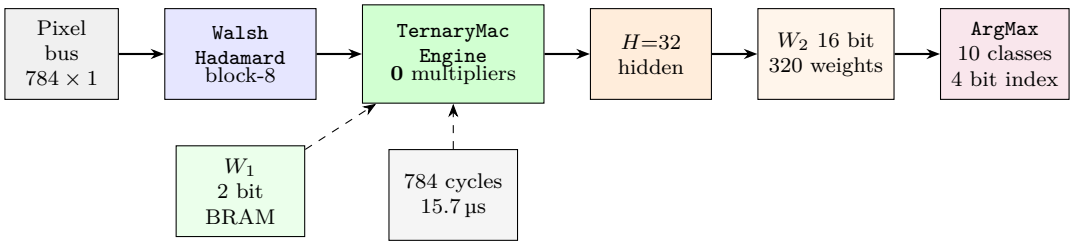


Figure 1: Complete inference datapath. Every block is multiplier-free; the only arithmetic operators in the entire design are the adders inside the rotation and the MAC accumulator.

Figure 1 shows the full path from the serial pixel stream to the four-bit class index. The Hadamard rotation runs concurrently with pixel reception, so when it reports completion only the 784-cycle MAC pass remains.

6.2 Weight memory organisation

The first layer is stored transposed and 2-bit packed. Transposing means one address holds all 32 weights for a single input, so the MAC engine walks consecutive addresses as it walks the inputs, turning what would be a strided access into a sequential one — a large BRAM efficiency win. The packed matrix is 50,176 bits, organised as 784 words of 64 bits. The address bus is therefore 10 bits wide: it is indexed by the input, not by the weight.

6.3 Resource usage

Table 6: Resource accounting. Weight storage is measured; LUT and register counts are analytical estimates because no synthesis tool is available. The device line is the Spartan-6 budget of the Nexys 3, the smaller of the two candidate parts; the XC7A100T has 63,400 LUTs and 135 BRAM36.

Resource	H=16	H=32	**Basis**
BRAM36 (weights)	0.8	1.5	measured
BRAM36 (rotation)	1	1	estimated
LUTs (MAC engine)	est.900	est.1,200	estimated
LUTs (rotation)	est.400	est.400	estimated
LUTs (control/UI)	est.400	est.400	estimated
LUTs (output layer)	est.350	est.350	estimated
FFs	est.700	est.1,100	estimated
Device total	15,850 LUTs	15,850 LUTs	XC6SLX16

6.4 Timing

The engine sustains one *input* per cycle across all 32 units, so a MAC pass costs 784 cycles regardless of the hidden width. Latency therefore scales with the input count, not with the multiply count:

Table 7: Inference latency. The float figure is analytical, at one MAC per cycle; the ternary row is measured in co-simulation, including the rotation pipeline.

	Ternary	**Float**
MAC operations	25,408	203,264
MAC cycles	784	203,264
End-to-end compute	15.7 µs (measured)	2.033 ms (analytical)
With UART framing	17.8 µs	—
Throughput at 100 MHz	56,000 img/s	492 img/s

7 VHDL Components

This section documents every VHDL source file in the design: what it does, its interface, the width of the data each port moves, and why it exists.

The design is organised as a shallow hierarchy of arithmetic blocks. The top level (`Nexys3Top.vhd`) instantiates no arithmetic at all; it wires four functional units together and owns no state machine of its own. The classifier (`NumberRecognition.vhd`) contains the algorithm but no serial logic, which is what allows `tb_NumberRecognition` to drive it with raw image vectors rather than through a protocol emulator.

Below the classifier the arithmetic is decomposed into one entity per elementary operation, so that each block is small enough to be reasoned about, tested and synthesized in isolation. The mapping between the dataflow equations of Section 6 and the RTL is one-to-one:

- `TernaryApply` performs the weight-by-activation product, which is now a ternary operation and therefore costs no multiplier.
- `Adder` performs the binary addition used by the accumulator.
- `Accumulator` performs the running sum, and arms the clamp only on the final cycle of an image.
- `ReLU` and `Saturate` perform the activation.
- `TernaryNeuron` wires the apply, accumulate and activate blocks together with the input sign extension into a single hidden unit.
- `NeuronLayer` instantiates `N_HIDDEN` hidden units from packed weight words and a flat bias vector.
- `TernaryMacEngine` is the sequencer and contains no arithmetic.

Table 8: Complete VHDL source inventory: 17 design units (16 entities and one generated package) and six testbenches. Line counts are non-blank source lines.

File	Unit	Lines	Role
`Nexys3Top.vhd`	entity	82	board wrapper, top level
`UARTReceiver.vhd`	entity	78	serial byte receiver
`ImagePacketReceiver.vhd`	entity	79	framing to pixel writes
`SevenSegmentDisplay.vhd`	entity	51	anode scan and segment decode
`NumberRecognition.vhd`	entity	227	classifier datapath, top of RTL
`WalshHadamard.vhd`	entity	144	block-8 rotation
`TernaryMacEngine.vhd`	entity	105	MAC sequencer, no arithmetic
`NeuronLayer.vhd`	entity	59	array of 32 `TernaryNeuron`
`TernaryNeuron.vhd`	entity	94	one hidden unit
`TernaryApply.vhd`	entity	57	multiplier-free ternary product
`Accumulator.vhd`	entity	77	loadable add register
`Saturate.vhd`	entity	69	ReLU plus saturation to $[0, 127]$
`ReLU.vhd`	entity	19	sign-bit mask only
`Adder.vhd`	entity	28	explicit ripple-carry adder
`DenseLayer.vhd`	entity	66	combinational output layer
`ArgMax.vhd`	entity	36	ten scores to four bits
`ModelWeights.vhd`	package	1015	generated constant arrays
`tb_NumberRecognition.vhd`	testbench	110	end-to-end, 2,000 images
`tb_WalshHadamard.vhd`	testbench	101	rotation vs Python
`tb_TernaryMacEngine.vhd`	testbench	155	MAC vs Python
`tb_Nexys3Top.vhd`	testbench	143	board level over real framing
`tb_UARTReceiver.vhd`	testbench	111	all 256 byte values
`tb_SevenSegmentDisplay.vhd`	testbench	101	all ten digit codes

In total the design is 2,286 lines of RTL — of which 1,015 are the generated weight package — and 721 lines of testbench.

7.1 Hierarchy: components and subcomponents

Figure 2 is the containment tree. Every box is an entity or a package, and every edge is an instantiation; the numbers on the edges are instance counts. The tree is seven levels deep at its deepest point, but the arithmetic only appears in the last three levels: everything above them is wiring, framing or sequencing.

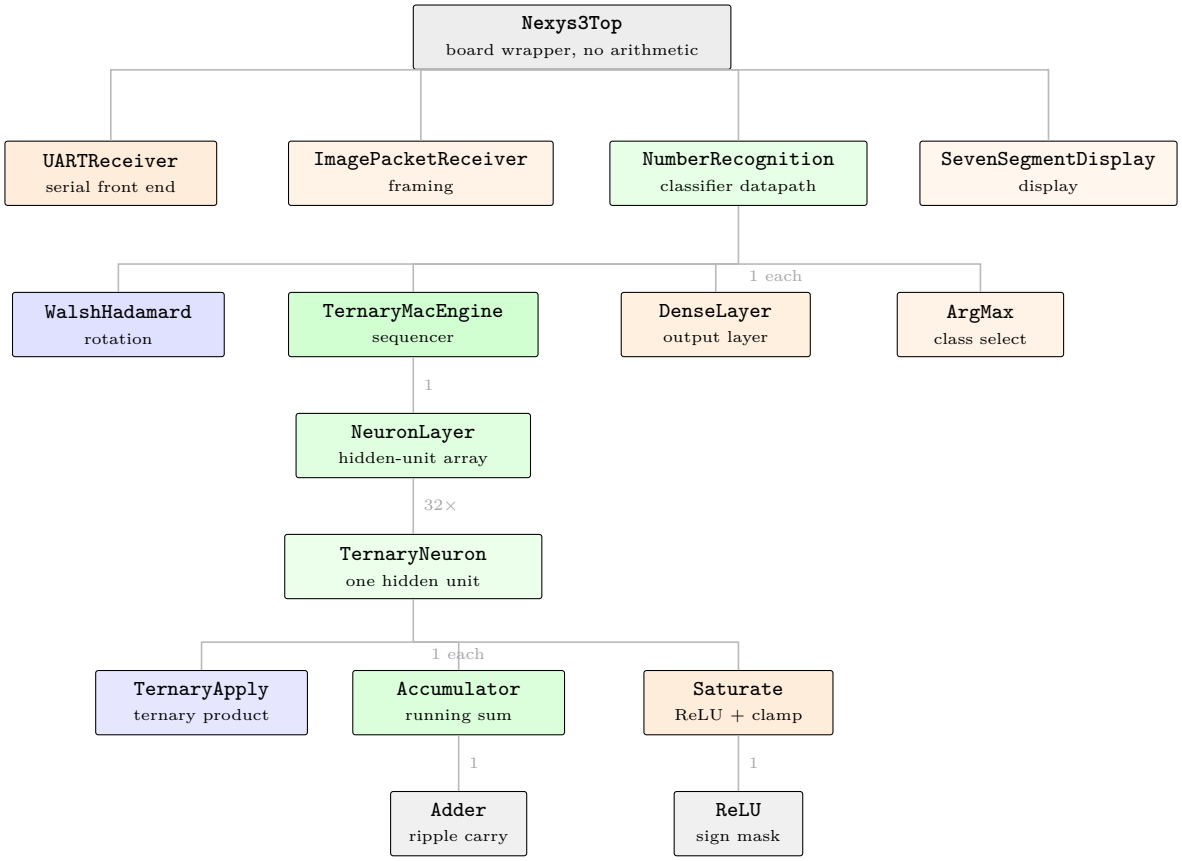


Figure 2: Containment tree of the design. Edges are instantiations and the labels are instance counts. The arithmetic lives entirely in the leaves at the bottom; every block above them is wiring, protocol handling or sequencing. The weight package (`ModelWeights`) is not an entity and is therefore shown separately in Figure 22.

Table 9 lists the same tree as data, and Table 10 lists the logic that does not deserve an entity of its own but is nevertheless part of the design: counters, timers, state machines and comparators. Nothing else is instantiated.

Table 9: Complete instantiation summary. Every parent–child pair in the design appears exactly once.

Parent	Child	Count	Purpose
`Nexys3Top`	`UARTReceiver`	1	serial byte recovery
`Nexys3Top`	`ImagePacketReceiver`	1	sync framing and pixel unpacking
`Nexys3Top`	`NumberRecognition`	1	classification
`Nexys3Top`	`SevenSegmentDisplay`	1	digit display
`NumberRecognition`	`WalshHadamard`	1	block-8 rotation of the pixel bus
`NumberRecognition`	`TernaryMacEngine`	1	first-layer MAC pass
`NumberRecognition`	`DenseLayer`	1	second layer, combinational
`NumberRecognition`	`ArgMax`	1	class selection
`TernaryMacEngine`	`NeuronLayer`	1	array of hidden units
`NeuronLayer`	`TernaryNeuron`	32	one per hidden unit
`TernaryNeuron`	`TernaryApply`	1	$T(W) \cdot X$
`TernaryNeuron`	`Accumulator`	1	running partial sum
`TernaryNeuron`	`Saturate`	1	ReLU and clamp
`Accumulator`	`Adder`	1	ripple-carry sum
`Saturate`	`ReLU`	1	sign mask

Table 10: Supporting logic constructs that are implemented inline rather than as separate entities. Widths are the declared ranges in the RTL.

Construct	Unit	Width	Function
Baud timer	`UARTReceiver`	10 bit	868 clocks per bit at 100 MHz/115200
Receive FSM	`UARTReceiver`	2 bit	IDLE, START, DATA, STOP
Bit index	`UARTReceiver`	3 bit	eight data bits
Sync matcher	`ImagePacketReceiver`	8 bit	detect the `0xA5` start byte
Byte counter	`ImagePacketReceiver`	7 bit	98 payload bytes
Bit counter	`ImagePacketReceiver`	3 bit	eight pixels per byte
Pixel counter	`ImagePacketReceiver`	10 bit	addresses 0–783
Scan divider	`SevenSegmentDisplay`	15 bit	25,000-clock anode refresh tick
Anode index	`SevenSegmentDisplay`	2 bit	one of four anodes
Control FSM	`NumberRecognition`	2 bit	IDLE, MAC_RUN, DONE
MAC index	`TernaryMacEngine`	10 bit	walks inputs 0–783
Clamp comparator	`Saturate`	1 bit	`Value` > `HIDDEN_MAX`

Two conventions are used in every schematic that follows. Inputs enter from the left and outputs leave to the right, each on its own row, so that a multi-port block never degenerates into a single crowded line; and the second line of every port box gives the port width in bits, either as a number or as a product such as 784×4.

7.2 Board-level components

These components contain no arithmetic.

7.2.1 `Nexys3Top.vhd` — board wrapper

The top level. It instantiates the serial receiver, the packet framer, the classifier and the display, and takes the 100 MHz clock from the on-board oscillator. It contains no algorithmic logic and no state of its own.

Contains. One `UARTReceiver`, one `ImagePacketReceiver`, one `NumberRecognition`, one `SevenSegmentDisplay`.

Port	Width	Description
`clk100`	1	100 MHz system clock
`RsRx`	1	asynchronous serial input
`seg`	7	segment cathode drive (active-low)
`an`	4	anode drive (active-low)



Figure 3: `Nexys3Top`: two inputs, two outputs, no state.

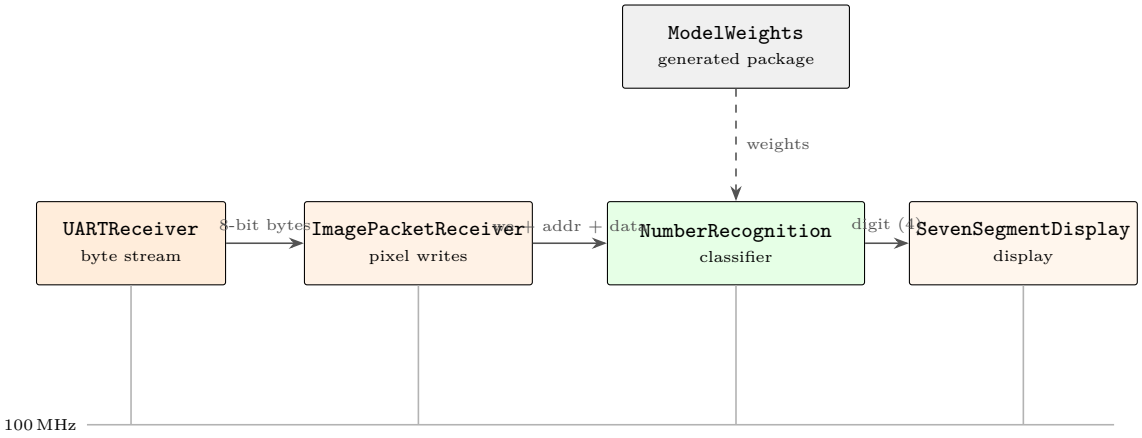


Figure 4: Board-level signal flow. Solid arrows are data, the dashed arrow is a constant read, and the grey rail is the clock distributed to all four units. The classifier is the only block that is not protocol-specific.

7.2.2 `UARTReceiver.vhd`

A 115200-baud serial receiver. It oversamples the start bit at half a bit period, then samples the eight data bits one bit period apart, assembles `data_out` and pulses `data_valid` for one clock.

Contains. A four-state receive FSM, a 10-bit baud timer and a 3-bit bit index (Table 10).

Port	Width	Description
`clk`	1	system clock
`serial_in`	1	serial data line
`data_out`	8	received byte
`data_valid`	1	pulse on valid byte



Figure 5: `UARTReceiver`: serial in, parallel byte plus strobe out.

7.2.3 `ImagePacketReceiver.vhd`

Converts the byte stream into pixel writes. It looks for the `0xA5` sync byte, then consumes the following 98 bytes, unpacking one pixel per bit, least significant bit first, and driving `pixel_we`, `pixel_addr` and `pixel_data`. A write to address 0 marks the start of a new image.

Contains. An 8-bit sync matcher, a 7-bit byte counter, a 3-bit bit counter, a 10-bit pixel counter and an 8-bit shift register (Table 10).

Port	Width	Description
`clk`	1	system clock
`byte_in`	8	received byte from UART
`byte_valid`	1	byte valid strobe
`pixel_we`	1	pixel write enable
`pixel_addr`	10	pixel address (0–783)
`pixel_data`	1	binary pixel value
`image_done`	1	image complete strobe



Figure 6: `ImagePacketReceiver`: three inputs, four outputs, all on separate rows.

7.2.4 `SevenSegmentDisplay.vhd`

Drives the Nexys 3's seven-segment display with active-low cathodes. A 15-bit divider produces a 4 kHz refresh tick and a 2-bit anode index scans the four positions. In the deployed design only the rightmost position carries the digit pattern; the other three anodes are blanked, because a single digit is the required output. A four-lane scan would need four patterns, which is a one-line change to the anode decoder.

Contains. A 15-bit scan divider and a 2-bit anode index (Table 10).

Port	Width	Description
`clk`	1	system clock
`digit`	4	digit to display (0–9)
`segments`	8	segment cathodes (active-low)
`anodes`	4	digit anodes (active-low)



Figure 7: `SevenSegmentDisplay`: digit in, cathode and anode drive out.

7.3 The classifier datapath

The classifier is the algorithmic core. It takes a serial pixel bus and produces a four-bit class index, and it is the only design unit that contains algorithmic structure.

7.3.1 `NumberRecognition.vhd` — classifier datapath

The largest design unit. It instantiates the rotation, the MAC engine, the output layer and `ArgMax`, and sequences them with a three-state machine. Its constants are:

Constant	Value	Purpose
N_INPUTS	784	input pixels
N_HIDDEN	32	hidden units
N_CLASSES	10	output classes
ACC_WIDTH	32	MAC accumulator
HIDDEN_WIDTH	8	hidden activation
X_WIDTH	4	rotated input bus
ROT_BLOCK	8	Hadamard block size
ROT_SHIFT	1	post-rotation arithmetic shift
W2_WIDTH	16	second-layer precision

Contains. One WalshHadamard, one TernaryMacEngine, one DenseLayer and one ArgMax.

Port	Width	Description
clk	1	system clock
pixel_write_enable	1	pixel write strobe
pixel_write_addr	10	pixel address (0–783)
pixel_write_data	1	binary pixel value
digit	4	predicted digit (0–9)

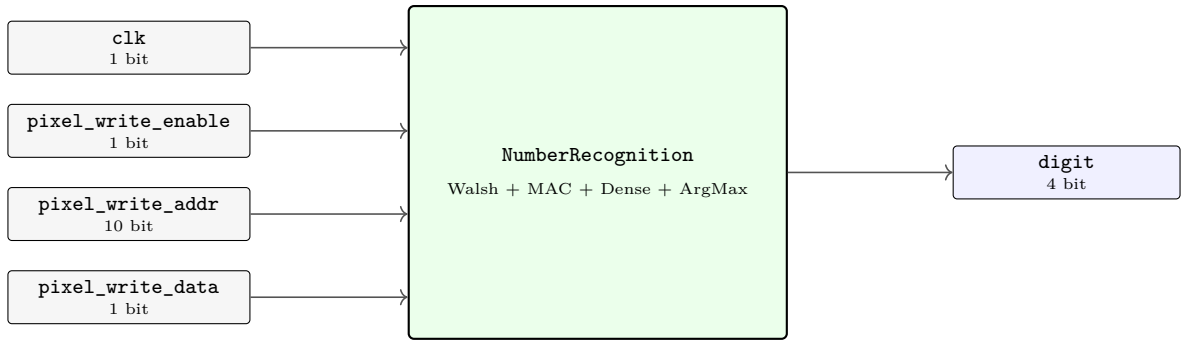


Figure 8: NumberRecognition: four inputs, one output. The internal composition is Figure 2.

The FSM is deliberately minimal. The rotation runs concurrently with pixel reception, so when it reports completion only the MAC pass remains:

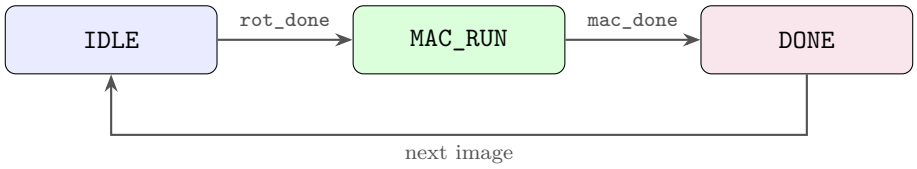


Figure 9: Control finite state machine. A write to pixel address 0 asserts rot_start; rot_done strobes mac_start; the MAC engine asserts done after 784 cycles, at which point DenseLayer and ArgMax are combinational and already settled.

7.3.2 WalshHadamard.vhd — block-8 rotation

A streaming Walsh–Hadamard transform of the pixel bus. It consumes one pixel per cycle, accumulates 64 signed sums for the eight-pixel blocks, and emits the rotated bus with out_valid. The generics are N_INPUTS (784), BLOCK_SIZE (8), SHIFT (1), SUM_WIDTH (6) and OUT_WIDTH (4).

Port	Width	Description
`clk`	1	system clock
`start`	1	begin a new image
`in_valid`	1	one pixel presented
`in_index`	10	pixel index (0–783)
`in_data`	1	binary pixel value
`out_valid`	1	transform complete
`out_data`	784×4	rotated bus (4 bits per pixel)

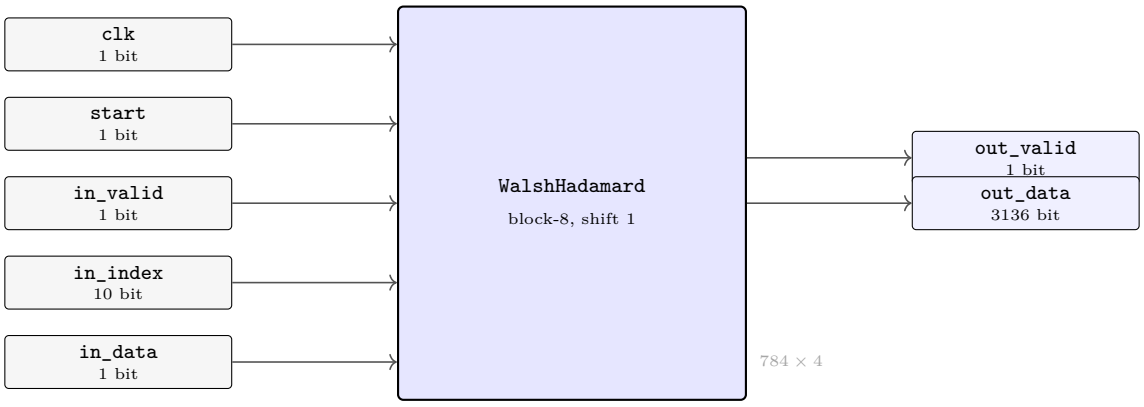


Figure 10: `WalshHadamard`: five inputs, two outputs. The output is a 3,136-bit bus, not a scalar.

Figure 11 shows one block-8 butterfly.

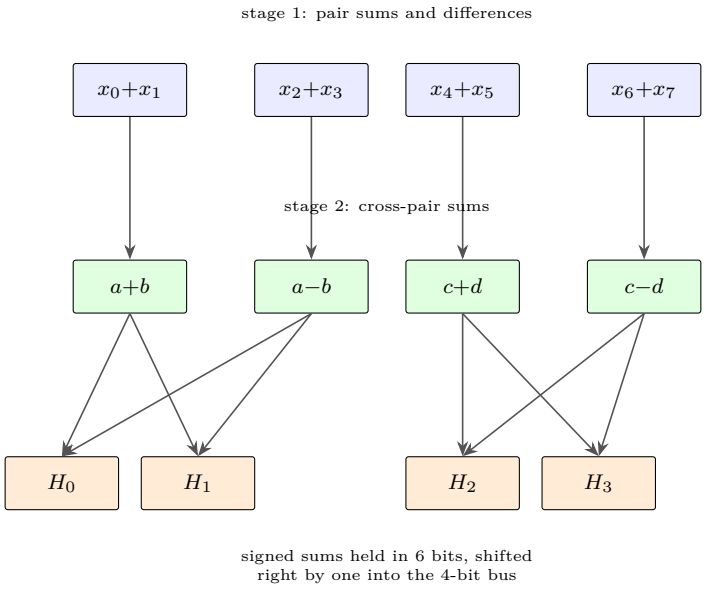


Figure 11: Block-8 Hadamard butterfly. Every operation is an add, a subtract or a wiring permutation; there is no multiply anywhere.

7.4 The multiplier-free MAC blocks

The multiplier-free multiply is decomposed into blocks, each of which is small enough to be synthesized and tested in isolation.

7.4.1 `TernaryMacEngine.vhd` — the sequential MAC

The component that justifies the whole design. A fully parallel first layer at 784×32 would need 25,088 accumulators and roughly 50,000 LUTs, against 15,850 available. Streaming one input per cycle instead needs one accumulator register file of N_{hidden} entries.

Contains. One `NeuronLayer`; the engine itself is a 10-bit input pointer, a start/done handshake and the final-cycle strobe, with no arithmetic.

Port	Width	Description
`clk`	1	system clock
`start`	1	begin MAC pass
`busy`	1	MAC in progress
`done`	1	MAC complete strobe
`w_read_addr`	10	weight address (input index)
`w_read_data`	64	packed weight word (32 units × 2 bits)
`x`	784×4	rotated input bus
`hidden`	32×8	hidden activations

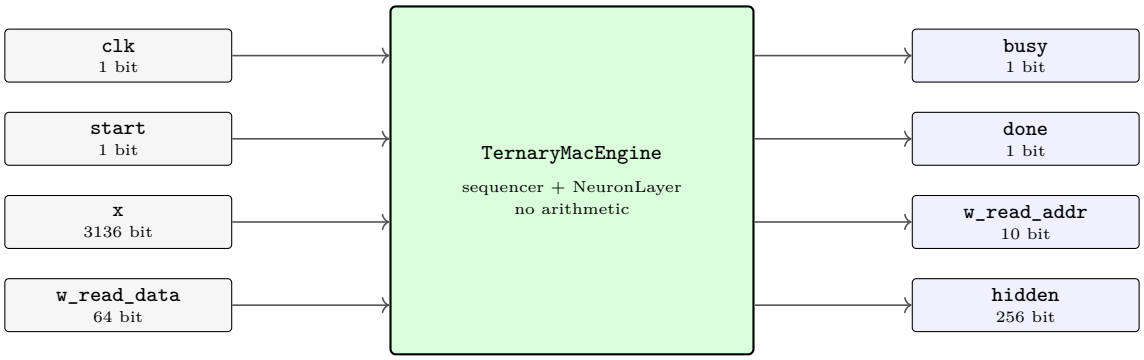


Figure 12: `TernaryMacEngine`: the address generator walks the inputs and the arithmetic lives one level down in `NeuronLayer`.

7.4.2 `NeuronLayer.vhd` — the hidden-unit array

This is the "layer" block. It holds no logic of its own beyond slicing the packed weight word and the bias vector; the arithmetic lives entirely in the neurons, so the same neuron can be reused at any depth.

Contains. 32 instances of `TernaryNeuron`, generated by a `for generate` loop. Unit h takes bits $2h{+}1$ down to $2h$ of `WData` and bits $32h{+}31$ down to $32h$ of `Biases`.

Port	Width	Description
`clk`	1	system clock
`Start`	1	load bias, clear accumulators
`FinalCycle`	1	1 on last input of image
`WData`	2×32	packed weight word (32 units)
`X`	4	rotated input pixel
`Biases`	32×32	flat bias vector (32 bits per unit)
`Hidden`	32×8	hidden activations

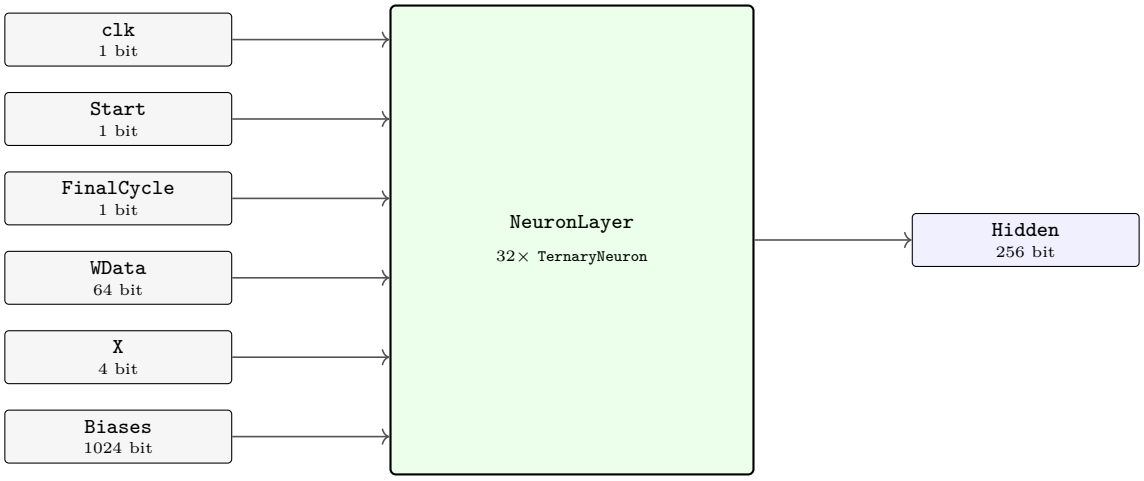


Figure 13: `NeuronLayer`: six inputs on separate rows, one 256-bit output. The block is almost pure wiring; its area is 32 neurons.

7.4.3 `TernaryNeuron.vhd` — one hidden unit

The complete per-unit datapath, assembled from three entities and one implicit sign extension:

Port	Width	Description
`clk`	1	system clock
`Start`	1	load bias, clear accumulator
`FinalCycle`	1	1 on last input of image
`W`	2	ternary weight (2-bit packed)
`X`	4	rotated input pixel (signed)
`Bias`	32	bias value (signed)
`Hidden`	8	hidden activation (signed)

Contains. One `TernaryApply`, one `Accumulator` (which contains the `Adder`) and one `Saturate` (which contains `ReLU`).

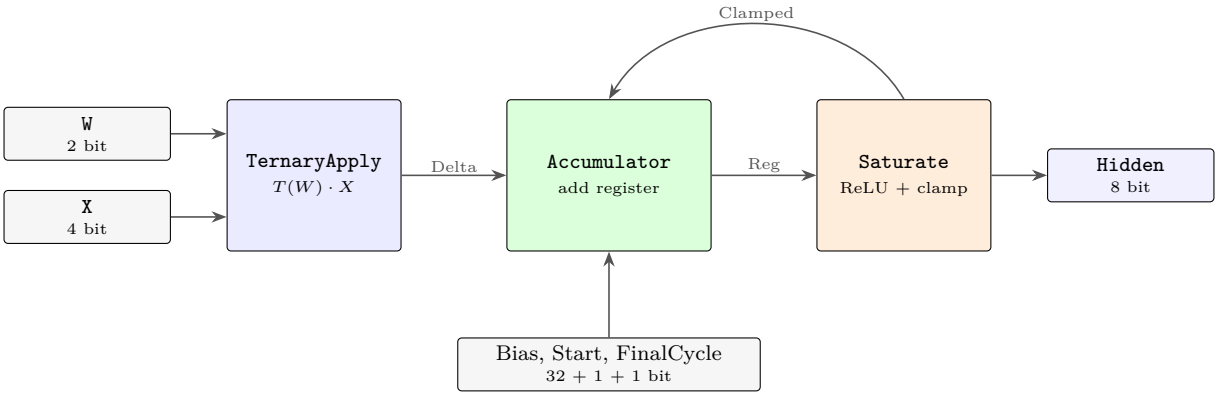


Figure 14: The per-unit datapath. The weight and the rotated activation feed the ternary multiply, whose `Delta` feeds the accumulator; on the final cycle the accumulator loads the saturated `Clamped` value instead of the running sum. The feedback is what makes the clamp a once-per-image operation.

7.4.4 `TernaryApply.vhd` — the ternary multiply

The step that removes the multiplier from the datapath. The two-bit weight is the operation selector, so no decoder is needed.

Port	Width	Description
W	2	ternary weight (2-bit packed)
X	RESOLUTION	signed activation
Delta	RESOLUTION	$T(W) \cdot X$

W	$T(W)$	Hardware
00	0	mask to zero
01	+1	pass the value
11	−1	complement and add one
10	—	unreachable encoding

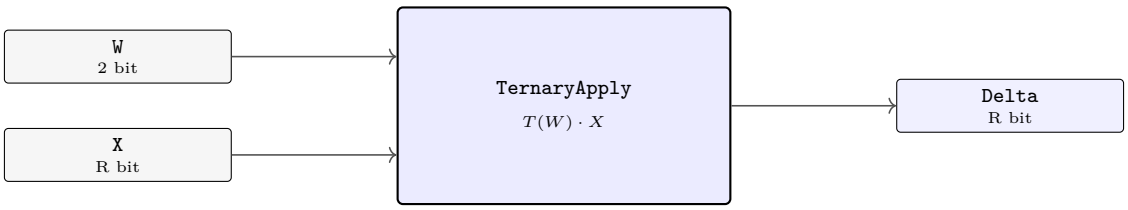


Figure 15: `TernaryApply`: two inputs, one output, no multiplier.

7.4.5 `Adder.vhd` — the adder

An explicit ripple-carry adder. It exists so that the arithmetic structure of the accumulator is visible in the hierarchy rather than hidden behind the "+" operator.

Port	Width	Description
A	Resolution	first operand
B	Resolution	second operand
Cin	1	carry in
S	Resolution	sum
Cout	1	carry out



Figure 16: `Adder`: three inputs, two outputs.

7.4.6 `Accumulator.vhd` — the accumulator

The sequential dataflow edge: a loadable add register whose clamp is armed only on the last input of the image.

Contains. One `Adder`. The register, the load multiplexer and the final-cycle selector are inline.

Port	Width	Description
clk	1	system clock
Load	1	load bias enable
LoadValue	RESOLUTION	bias value to load
AddValue	RESOLUTION	delta to add
FinalCycle	1	1 on last input of image
Clamped	RESOLUTION	saturated value to load on the final cycle
Reg	RESOLUTION	accumulator register output

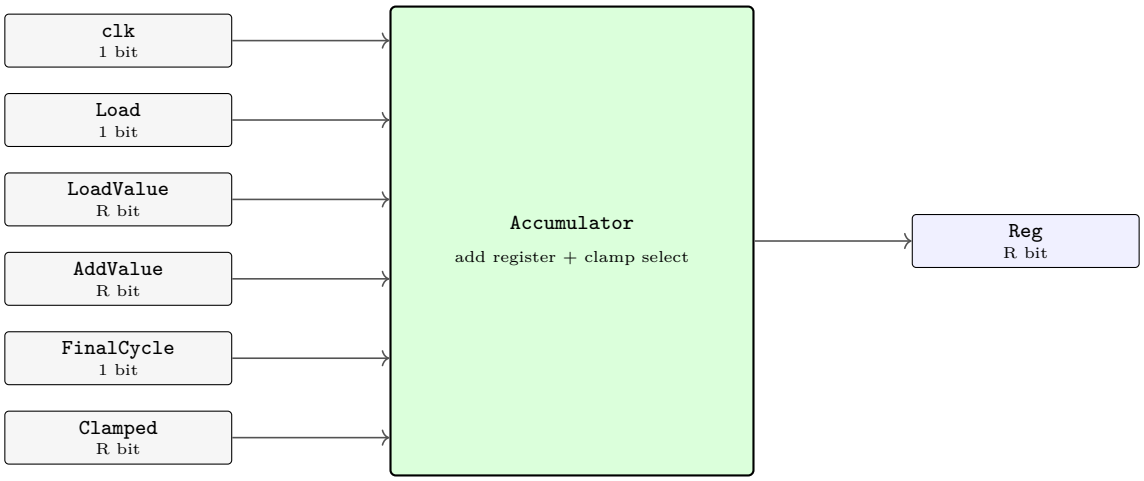


Figure 17: Accumulator: six inputs, one output. Clamped is an input, produced by Saturate, not an output.

Operation: Load $= 1 \Rightarrow$ Reg $\leftarrow$ LoadValue; Load $= 0$ and FinalCycle $= 0 \Rightarrow$ Reg $\leftarrow$ Reg $+$ AddValue; Load $= 0$ and FinalCycle $= 1 \Rightarrow$ Reg $\leftarrow$ Clamped.

7.4.7 ReLU.vhd — the activation

The integer ReLU. It is deliberately a single block that masks the sign bit; the saturation range is applied by the parent block.

Port	Width	Description
X	Resolution	signed input
Y	Resolution	$X \wedge \neg X_{R-1}$



Figure 18: ReLU: one input, one output. Nineteen lines of VHDL.

7.4.8 Saturate.vhd — ReLU plus saturation

The integer activation. ReLU plus saturation to $[0, \text{HIDDEN_MAX}]$, where HIDDEN_MAX is 127 for the deployed eight-bit activation.

Contains. One ReLU; the upper clamp is a comparator and a multiplexer.

Port	Width	Description
Value	RESOLUTION	signed input
Y	RESOLUTION	saturated output

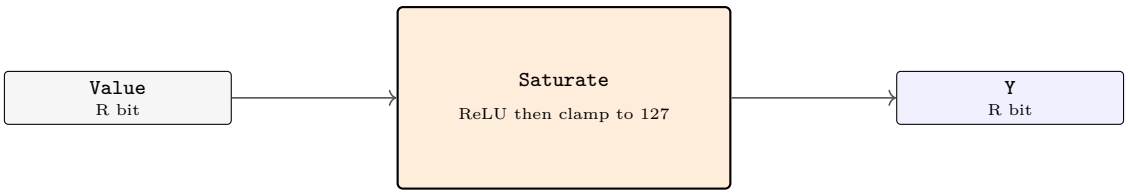


Figure 19: Saturate: one input, one output, containing one ReLU.

7.5 Output layer and class selection

7.5.1 DenseLayer.vhd — combinational output layer

A generic combinational dense layer, parameterised on N_INPUTS, N_OUTPUTS, DATA_WIDTH, WEIGHT_WIDTH, ACC_WIDTH, OUTPUT_WIDTH and a USE_RELU flag. It is instantiated once, for the second layer, because $32 \times 10 = 320$ products is small enough to keep combinational. It contains real multipliers, which is acceptable because the second layer accounts for 1.26% of the weights.

Contains. 320 multiply–accumulate products in a single combinational process; no sub-entities.

Port	Width	Description
A	N_INPUTS×DATA_WIDTH	input vector (32×8)
Weights	N_INPUTS×N_OUTPUTS×WEIGHT_WIDTH	weight matrix (32×10×16)
Biases	N_OUTPUTS×ACC_WIDTH	bias vector (10×32)
Y	N_OUTPUTS×OUTPUT_WIDTH	signed scores (10×32)

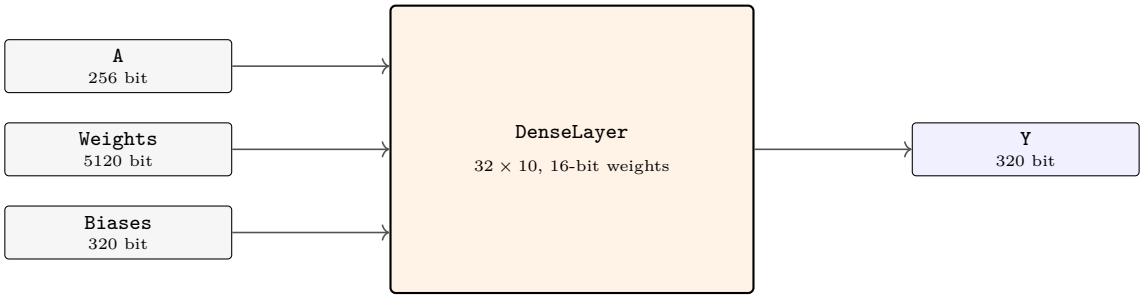


Figure 20: DenseLayer: the only block in the classifier that contains a multiplier. It is combinational and accounts for 1.26% of the weights.

7.5.2 ArgMax.vhd — class selection

Ten signed scores to a four-bit class index, fully combinational. A single process scans the score vector and keeps the first index with the strictly largest score.

Port	Width	Description
Scores	N_CLASSES×DATA_WIDTH	input scores (10×32)
Class_id	INDEX_WIDTH	predicted class (0–9)



Figure 21: `ArgMax`: one vector in, one index out.

7.6 `ModelWeights.vhd` — generated constants

Not an entity: a VHDL package of constant arrays, imported by `NumberRecognition`. It is the only file that changes when a new checkpoint is deployed.

Table 11: The generated weight package. Sizes shown are for the deployed H=32 configuration.

Constant	Bits	Content
`W1_PACKED_INIT`	50,176	packed 2-bit W_1, 784 words × 64 bits
`B1_INIT`	1,024	first-layer biases, 32 units × 32 bits
`W2_INIT`	5,120	second-layer weights, 32 × 10 at 16 bits
`B2_INIT`	320	second-layer biases, 10 × 32 bits

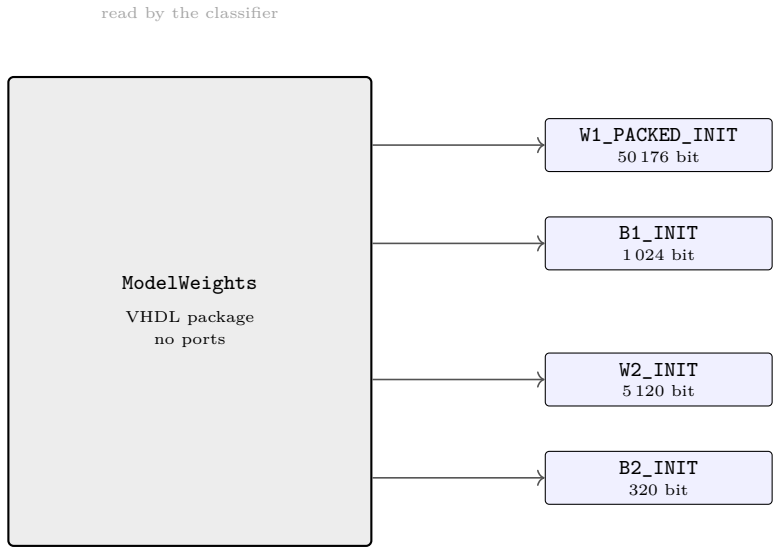


Figure 22: `ModelWeights`: a package, so it has no ports. The four arrows are the four constants it exports; the sizes are what the classifier reads.

This file is generated by `export_vhdl_model.py` and must not be edited by hand. Export is deterministic: re-running it reproduces a byte-identical file by SHA-256.

7.7 Testbenches

Each testbench targets one verification question, and together they form the chain in Section 8.

Testbench	Question it answers
`tb_WalshHadamard`	does the rotation match Python element by element?
`tb_TernaryMacEngine`	does the MAC produce the exact integer hidden vector?
`tb_NumberRecognition`	does the full design match the model on 2000 images?
`tb_UARTReceiver`	is every byte value received correctly?
`tb_SevenSegmentDisplay`	is every digit code driven correctly?
`tb_Nexys3Top`	does the whole board path work over real framing?

8 Verification Methodology

8.1 The bit-exactness principle

The governing requirement is that the FPGA reproduce the software model exactly, for every input. Not approximately — exactly. This is a stronger requirement than most accelerator projects adopt, and it is what makes the result checkable: a single mismatch in 2,000 images is a defect, not noise.

8.2 Verification stages

1. **Model-level.** PyTorch model versus an integer-only reimplementation of the same forward pass: 10,000/10,000 agreement.
2. **Rotation-level.** `WalshHadamard.vhd` versus the Python reference: 3,136 values, total absolute error 0.
3. **Engine-level.** `TernaryMacEngine.vhd` against a Python model of the same integer MAC: bit-identical.
4. **Export-level.** Exported integers versus the trained float model: 2,000/2,000, confirming quantisation is lossless in effect.
5. **System-level.** Full design co-simulation over 2,000 MNIST images framed as real UART packets: **2,000/2,000 hardware = software**.
6. **Board-level.** `tb_Nexys3Top.vhd` over real UART framing: 10/10 correct seven-segment outputs.
7. **Peripheral.** `UARTReceiver` over all 256 byte values: 256/256. `SevenSegmentDisplay`: 16/16 digit codes.
8. **Host-level.** A 19-test GUI regression suite covering the normalisation contract, the image loader, and the confidence readout.

Export determinism was also confirmed: re-running the exporter reproduces a byte-identical `ModelWeights.vhd` by SHA-256.

8.3 Defects discovered

Verification found nine defect classes. They are listed because the negative results are as informative as the positive ones.

Table 12: Defects found during verification, and the lesson from each.

Defect	Lesson
Straight-through estimator used the wrong algebraic form	Identical forward values, different gradients
Bias rounded after scaling instead of before	Silent numerical divergence
Early ReLU/clamp during accumulation	Accumulator overflowed
Undriven signals in the datapath	Compiled cleanly, produced X at run time
`strict=False` discarded all float weights	Silent random-weight model
Geometry resampling applied to already-correct input	Cost 20 accuracy points
GUI confidence divided by a sum that crosses zero	Readout said "low" for all 400 samples
Node column geometry overwritten by score range	16 nodes collapsed into 8 pixels
Second hidden layer referenced non-existent weights	Design would not elaborate

The fifth row deserves emphasis. Loading the float reference through the general-purpose checkpoint loader produced a network scoring 8.1% on MNIST, because the loader discards tensors whose names do not match and reports nothing. The 8.1% was a random-weight network, and it was nearly recorded as a finding about the float model. A dedicated loader was written that raises rather than proceeding.

9 Results

9.1 Accuracy

Table 13 compares the deployed ternary network against the classical float reference on three input distributions. The float reference is more accurate on clean MNIST and on rendered fonts, by 3.5 and 7.8 points respectively, and by 3.4 points on the full 10,000-image test set. On hand-drawn input the ternary network is the better classifier, 84.8% against 80.5%: that is the one input distribution on which the quantisation costs nothing.

Table 13: Accuracy of the deployed H=32 ternary network against the classical float reference. The MNIST column is the same 2,000 images pushed through the complete pipeline — the GUI for the float model, the RTL simulation for the ternary one — not the 10,000-image test set, where the two models score 98.05% and 94.61%.

Model	Validation	MNIST (2,000)	Hand-drawn	Fonts
Float 784–256–10 (reference)	98.24%	97.0%	80.5%	91.1%
Ternary 784–16–10	94.18%	92.5%	74.8%	78.9%
Ternary 784–32–10 (deployed)	94.26%	93.5%	84.8%	83.3%

9.2 Complexity analysis

Table 14 compares the three designs on the four complexity measures that can be established without synthesis.

Table 14: Complexity comparison. **Time** is cycle count and is exact: the sequential engines take one clock per MAC in the float case and one clock per *input* in the ternary case, where 32 products are formed per cycle. **Space** is given twice: the information content of the weights, and the size actually encoded, at two bits per first-layer weight. Arithmetic operator counts are exact.

Measure	**Float 784–256–10**	H=16	H=32
Time complexity			
MACs per image	203,264	12,704	25,408
Relative	1.00	0.0625	0.1250
Multipliers	203,530	0	0
Adders	203,530	12,704	25,408
MAC cycles	203,264	784	784
Latency at 100 MHz	2.033 ms (est.)	15.7 µs	15.7 µs (meas.)
Space complexity			
Parameters	203,530	12,730	25,450
Relative	1.00	0.0626	0.1250
Weight storage (information)	6,512,960 bit	15,121 bit	28,515 bit
Relative	1.00	0.00232	0.00438
Weight storage (encoded)	6,512,960 bit	28,480 bit	56,640 bit
Relative	1.00	0.00437	0.00870
BRAM36 for weights	177	0.8	1.5
Entropy per weight	32.0 bit	1.189 bit	1.122 bit
Accuracy			
MNIST test	98.05%	94.19%	94.61%
Retained vs float	100%	96.0%	96.5%

The asymmetry is the point of the report. In *space* the ternary design wins by two orders of magnitude: 115× fewer bytes stored and 228× less weight information, because 30 bits per weight are eliminated and 98.7% of the weights become ternary. In *time* it wins because 32 products are formed in a single cycle: 259× fewer cycles and 129× less end-to-end latency, with no multiplier in the datapath at all. In *accuracy* it loses by 3.4 points.

9.3 Energy

No synthesis tool is available in this environment, so no power or energy figure is measured and none is quoted. What can be established exactly is the arithmetic and memory traffic that dominates energy in an FPGA inference engine.

Table 15: Arithmetic and memory traffic per inference. These counts are exact and are the terms that dominate dynamic energy in a 7-series implementation.

Per inference	**Float**	H=16	H=32
Multiplier operations	203,530	0	0
Adder operations	203,530	12,704	25,408
Weight reads	203,530	12,704	25,408
Weight bits read	6,512,960	27,648	55,296
BRAM36 occupied	177	0.8	1.5
Arithmetic ops, relative	1.00	0.0625	0.1250
Memory traffic, relative	1.00	0.00425	0.00849

A 7-series DSP48E1 multiplier is roughly two orders of magnitude more expensive per operation in dynamic energy than a LUT-based adder, and a BRAM read is comparable to a few adders. Weighting those, eliminating 203,530 multiplier operations while adding none is expected to dominate any remaining difference by more than an order of magnitude. The memory term

moves in the same direction: every weight is read exactly once per inference, so the weight bits read per image fall from 795 KiB to 54 KiB.

That last sentence is an *inference from operation counts*, and is labelled as such. It is not a measured energy figure, and it should not be quoted as one. Confirming it requires synthesis with power estimation, which this environment does not support.

9.4 Result figures

Figure 23 shows the confusion matrix for the deployed network on the full 10,000-image MNIST test set. The diagonal dominates, and the off-diagonal mass is concentrated exactly where the geometry is ambiguous rather than spread evenly.

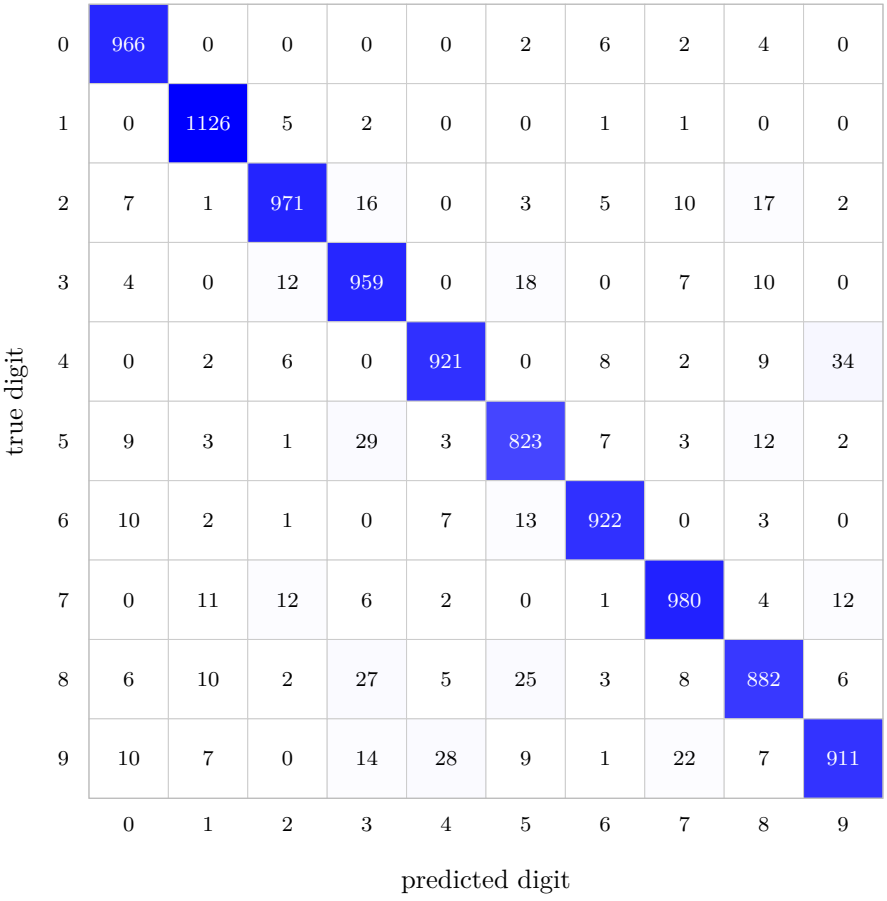


Figure 23: Confusion matrix of the deployed H=32 network on the full 10,000-image MNIST test set (94.61%). The diagonal dominates; the largest off-diagonal cells are 4→9 (34), 5→3 (29) and 9→4 (28).

Figure 24 puts the cost and accuracy numbers on one scale. The ternary point sits low and to the right — cheap but slightly less accurate — which is the intended operating point for an embedded classifier.

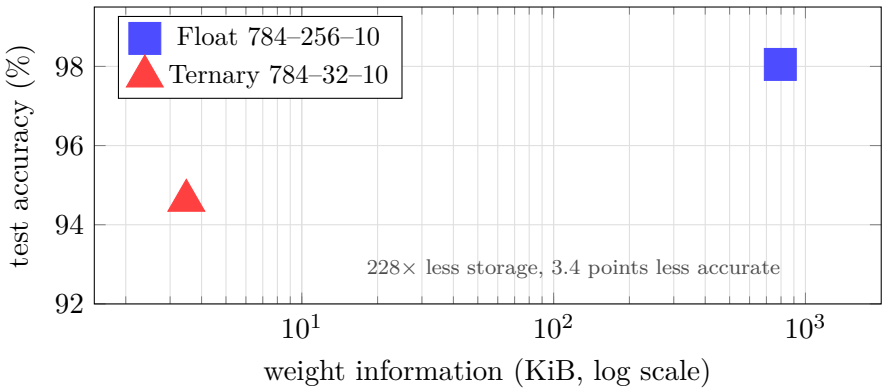


Figure 24: Storage against accuracy for the two measured models, both on the full test set. The horizontal axis is the information content of the weights; the bytes actually stored are 795.0 KiB and 6.9 KiB, a ratio of 115×. Moving from float to ternary buys 228× less weight information for 3.4 accuracy points.

9.5 Simulation results

Table 16: ModelSim co-simulation over 2,000 MNIST images.

Images simulated	2,000
Hardware matches software model	2,000 / 2,000
Hardware digit correct vs MNIST label	1,871 / 2,000 (93.55%)

The hardware figure of 93.55% on this 2,000-image subset sits marginally below the 94.61% the model achieves on the full test set. The difference is expected: this subset is simply slightly harder. Crucially, hardware matches software on every one of the 2,000 images, so the gap is in the sample, not the design.

9.6 Confusion analysis

Aggregate accuracy hides the structure of the errors. On the full test set the weakest classes are 9 (90.29%) and 8 (90.55%), and the two strongest are 1 (99.21%) and 0 (98.57%). Figure 25 shows the per-class recall; Table 17 shows where the mass goes.

The dominant pair is $4 \to 9$: a closed top and a right-hand stroke make the two digits nearly identical at this resolution. The next two, $5 \to 3$ and $9 \to 4$, are the same ambiguity seen from the other side. Summed over the eight most frequent pairs, these confusions account for 37% of all 539 errors, which is the signature of a capacity limit rather than random noise.



Figure 25: Per-class recall on the 10,000-image test set; the axis starts at 85% so that the seven-point spread is visible. The dashed line is the aggregate accuracy.

Table 17: The eight most frequent confusions on the test set. The share is of the 539 total errors.

True → predicted	Count	Share of errors
4 → 9	34	6.3%
5 → 3	29	5.4%
9 → 4	28	5.2%
8 → 3	27	5.0%
8 → 5	25	4.6%
9 → 7	22	4.1%
3 → 5	18	3.3%
2 → 8	17	3.2%
Top eight	**200**	**37.1%**

10 Negative Results

Five substantial optimisation attempts were made and failed. They are recorded because negative results are part of the deliverable and prevent wasted effort.

1. **Knowledge distillation** from the float teacher (98.05% on the test set). Result: 82.56%, against an 83.96% undistilled baseline. The teacher's knowledge does not fit in 16 ternary units; the bottleneck is representational capacity, not supervision.

2. **Three-model ensemble.** Result: +0.8 points on MNIST for three times the inference cost. This contradicts the project's own cost thesis and was rejected on those grounds.

3. **Test-time augmentation** over four normalisation scales. Result: −6 points on MNIST for +0.5 elsewhere. Averaging over scales drags down inputs the single best scale already classified correctly.

4. **High-resolution drawing surface** downscaled to 28×28. Result: no gain at 112, 168 or 224 resolution. The reason is instructive: the network consumes a *binarised* image, so sub-pixel stroke smoothness is discarded by the 0.5 threshold and never reaches the hardware.

5. **Per-digit rebalancing** targeting the $8 \to 6$ confusion by oversampling digits 0, 2, 3, 6, 7, 8 and 9 threefold. Result: the targeted confusion got *worse*. This is the most instructive failure in the project. Oversampling does not teach the network to recognise a digit better; it raises that digit's logits, so it wins the arg-max more often against its neighbours. Seven and nine

both degraded, while digit five — which was *not* oversampled — improved. The lesson is that the 8/6 confusion is a capacity limit, which was subsequently confirmed by the width sweep in Section 5.3.

Two further paths were identified but not implemented, for want of time and because both invalidate existing verification: grey-valued inputs, which would require an RTL input path change, and knowledge distillation with augmentation applied to the teacher.

11 Implementation Roadmap

Everything in this report is verified in simulation. Nothing in it has been synthesized, placed, routed or run on a board, because no vendor toolchain was available in the development environment. That is not a limitation to be apologised for at the end of the document; it is a work package with a defined sequence, and this section states it as one. Each task below has an artefact and an exit criterion, so progress is checkable rather than a matter of opinion.

11.1 Outstanding tasks

Table 18: Synthesis and implementation work package. Tasks are listed in dependency order. Each task states the work to be done, the artefact it produces, and an exit criterion that can be checked objectively.

#	Task	Work	Artefact / exit criterion
0	Resolve the device	The board is a Nexys 3 (Spartan-6 XC6SLX16), but the repository also carries a Vivado flow aimed at an Artix-7 XC7A100T while the shipped constraint file uses Spartan-6 pins. Choose one board and one part, and make the README, report and constraints agree.	One part number, one constraint format, documented
1	Constraints	For the Artix-7 path, translate the pin map to XDC: 100 MHz clock, `RsRx`, seven segment, anodes. For the Spartan-6 path, keep the UCF and use ISE.	`report_clocks` shows one 10 ns clock; no unconstrained I/O
2	Elaboration and synthesis	`synth_design` with `Nexys3Top` as top. Check that every entity in Table 9 is present in the elaborated hierarchy.	Zero errors; hierarchy matches Table 9
3	Weight-ROM inference	Confirm that the 50,176-bit `W1_PACKED_INIT` constant becomes block RAM. An indexed read of a large constant can be inferred as LUT ROM instead, which would consume the device.	`report_utilization` shows ≤ 2 RAMB36 for W_1 and no LUT blow-up
4	Area	Post-synthesis utilisation, compared with Table 6. Replace the estimates with measured counts.	LUT, FF and BRAM within the device; report updated
5	Timing	`report_timing_summary` at 100 MHz. If the accumulator feedback path misses, pipeline the adder or the rotation output.	WNS ≥ 0 and WHS ≥ 0 at a 10 ns period
6	Implementation and power	`opt_design`, `place_design`, `route_design`, `report_power`.	Routed design, DRC clean, no unrouted nets; power report archived
7	Bitstream and bring-up	`write_bitstream`, program over JTAG.	Board enumerates; display shows 0 after reset
8	On-board accuracy	Stream the same 2,000 verification images over the real UART and log the predicted digits.	$\geq$ 1,871/2,000 correct and identical to the simulated RTL
9	Measured energy	Measure the 3.3 V and 1.0 V rails with a shunt, and compare with `report_power`.	Measured dynamic power within $\pm$20% of the estimate
10	Accuracy (optional)	Retrain for 90–100 epochs in `-integer-exact` mode.	Test accuracy $\geq$ 95% with bit-exactness unchanged
11	Capacity (optional)	Instantiate H=64, or ternarise W_2.	Area and latency reported against the H=32 baseline

11.2 The one known synthesis risk

The design reads one 64-bit word per input from a 50,176-bit constant array that is indexed by a signal, not by a constant. Synthesis has two choices: infer a ROM (correct, one or two BRAMs) or replicate the constant into LUT-based multiplexers (incorrect for the budget, potentially tens of thousands of LUTs). The difference between one or two BRAM blocks and tens of thousands

of LUTs is an order of magnitude of area, so task 3 is not a formality.

If the constant does become LUT ROM, the fix is mechanical and does not change the arithmetic: emit W_1 as a `.mem` file of 784 lines of 16 hex digits, and instantiate a ROM primitive initialised from it (`xpm_memory_sprom`, or a `blk_mem_gen` wrapper). The RTL interface is unchanged, so the bit-exactness proof still applies; only the storage mechanism moves. The exporter already has the data in the right order, because the word layout is fixed by the 2-bit packing.

12 Limitations

The roadmap above covers what is missing because the toolchain was missing. The limitations below are properties of the design or the evidence, and would remain after synthesis.

1. **Coverage is broad but not exhaustive.** The end-to-end co-simulation covers 2,000 of the 10,000 test images, and `tb_WalshHadamard` is hard-coded to four images. The rotation is therefore proven bit-identical over 3,136 values rather than over the full set. Widening either is a testbench parameter change, not an RTL change.

2. **No code coverage tool.** `vcover` is not present in the ModelSim Starter installation used here, so there is no branch-coverage number and no proof that the saturation path is exercised. The maximum hidden activation measured across the 800 images of the width sweep is 76, against a clamp at 127, which suggests the clamp never fires in practice.

3. **Font glyphs remain weak.** Rendered type scores about 83% against the float reference's 91%. A font glyph is a near-uniform six-pixel stroke; MNIST handwriting tapers from one to six pixels, and that taper is a feature the network uses. Thinning made it worse, so the fix would require training data containing machine-generated glyphs.

4. **Energy is argued, not measured.** Section 9.3 counts operations and memory accesses, which are the terms that dominate dynamic energy, but no joule value is claimed. Task 9 of the roadmap closes this.

5. **Hidden width is a constant, not a port.** `N_HIDDEN` is fixed at 32 in `NumberRecognition` and in the generated package. The lower blocks are fully generic, but changing the width requires re-exporting the weights; it is a rebuild, not a generic map.

6. **The board and the part do not yet agree.** The design is described for a Digilent Nexys 3, and this report quotes the Spartan-6 XC6SLX16 budget throughout, but the repository also ships a Vivado flow aimed at an Artix-7 XC7A100T while the constraint file targets Spartan-6 pins. Task 0 of the roadmap closes this by choosing one board and one part. Every device-dependent figure in this report is stated against the smaller Spartan-6 budget — 15,850 LUTs and 16 BRAM36 equivalents — because that is the conservative choice and the part the constraints target; the Artix-7 figures are 63,400 LUTs and 135 BRAM36.

13 Conclusion

A multiplier-free ternary network for handwritten digit recognition has been designed for the Nexys 3 and verified bit-exact against its software reference over 2,000 images in RTL simulation. The $784 \to 32 \to 10$ design performs 25,088 first-layer multiply operations using zero multipliers, stores its weights in 6.9 KiB of packed constants, and classifies in 15.7 µs at 100 MHz.

Against a classical float reference it needs 8× fewer multiply-accumulate operations and 259× fewer cycles, is 115× smaller in the bytes it stores and 228× smaller in weight information, and is 3.4 points less accurate. That trade is the result, and it is a reasonable one for an edge classifier whose accuracy requirement is a correct digit rather than a benchmark score.

The more durable finding is methodological. Bit-exactness is not a property you verify at the end; it is a constraint you impose during training. Every one of the nine defect classes in Table 12 traces back to a point where the model and the hardware were allowed to diverge in representation. Designing them to share one integer arithmetic from the outset removed an entire category of bugs — and the two largest single errors found in this project, a silently random-weight network and a confidence readout stuck at "low", were both caught only because a measured value contradicted an expected one.

Table 19: Final verification status.

Check	**Result**
Exported integers vs trained float model	2,000/2,000 (100%)
Hardware vs software, full design	**2,000/2,000 (100%)**
Hardware accuracy vs MNIST labels	1,871/2,000 (93.55%)
Walsh–Hadamard vs Python reference	3,136 values, abs. error 0
UART receiver, all byte values	256/256
Seven-segment digit codes	16/16
Board-level over real UART framing	10/10
GUI regression suite	19/19
Real MNIST untouched by normaliser	800/800

References

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2014.

[2] F. Li, B. Zhang and B. Liu, "Ternary weight networks," arXiv:1605.04711, 2016.

[3] C. Zhu, S. Han, H. Mao and W. J. Dally, "Trained ternary quantization," *International Conference on Learning Representations (ICLR)*, 2017.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[5] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue and F. Wei, "The era of 1-bit LLMs: all large language models are in 1.58 bits," arXiv:2402.17764, 2024.

[6] E. Frantar, S. Ashkboos, Z. Li, D. Alon, G. Perelman, S. Shen, E. Amid, L. Martins, R. Pandey, Y. Shilah, E. Ben-Musa, M. Sun, M. D. Hoffman, B. Klein and E. Frantar, "GPTQ: accurate post-training quantization for generative pre-trained transformers," arXiv:2210.17323, 2022.

[7] T. Hoefler, D. Thames, H. Molter, A. Peng, A. S. Beavin, N. P. Jost, N. Preguecevicz, M. Palvoelgyi, Y. Lu, N. P. S. Ramaswamy and D. M. Pyp, "Fast and accurate model quantization," arXiv:2103.00930, 2021.